University of Chittagong

Dept. of Computer Science & Engineering

DBMS Assignment

Tutorial-02

Course Title: Database Management Systems

Course Code: CSE-414

Submitted to

Course Instructor

Dr.Rudra Pratap Deb Nath

Professor

Dept. of Computer Science & engineering

University of Chittagong

Submitted by

Team Name: ByteForge

Department: Computer Science and Engineering

Semester: 4th Semester

Submission Date:

April 15, 2026

Task:01

1.Document and describe your database in the way you would do it (or have done it) in a project report.

Answer:

(I)Introduction: This database is designed to store and manage medical information related to drugs,disease and their relationships.The system is designed to support queries that analyze drug–disease relationships, identify common side effects, and explore clinical trial data efficiently so that users can retrieve meaningful insights about drugs and their applications.

(II)Scoop of the Database: The database includes the following types of information:

1.Drugs and their categories.

2.Diseases and their categories.

3.Side effects and Interactions of Drugs.

4.Commercial Drug Products and Companies.

5.Clinical trial information and their researcher and location.

(III)Assumption: To simplify the design, the following assumptions are made:

1.A drug can treat multiple diseases.

2.A disease can be treated by multiple drugs.

3.A drug can have multiple side effects.

4.A side effect can be caused by multiple drugs.

5.Drug interactions occur between two or more drugs.

6.A drug may have multiple commercial products.

7.A drug may be involved in multiple clinical trials.

(IV)Entities & Relationship

Entities:

1.Drug

2.Disease

3.Side-effect

4.Clinical Trial

5.Researcher

6.product

7.Company

Relationship:

1.Drugs and Diseases(M:N)

2.Drugs and Side Effects(1:N)

3.Drugs and Clinical Trials(M:N)

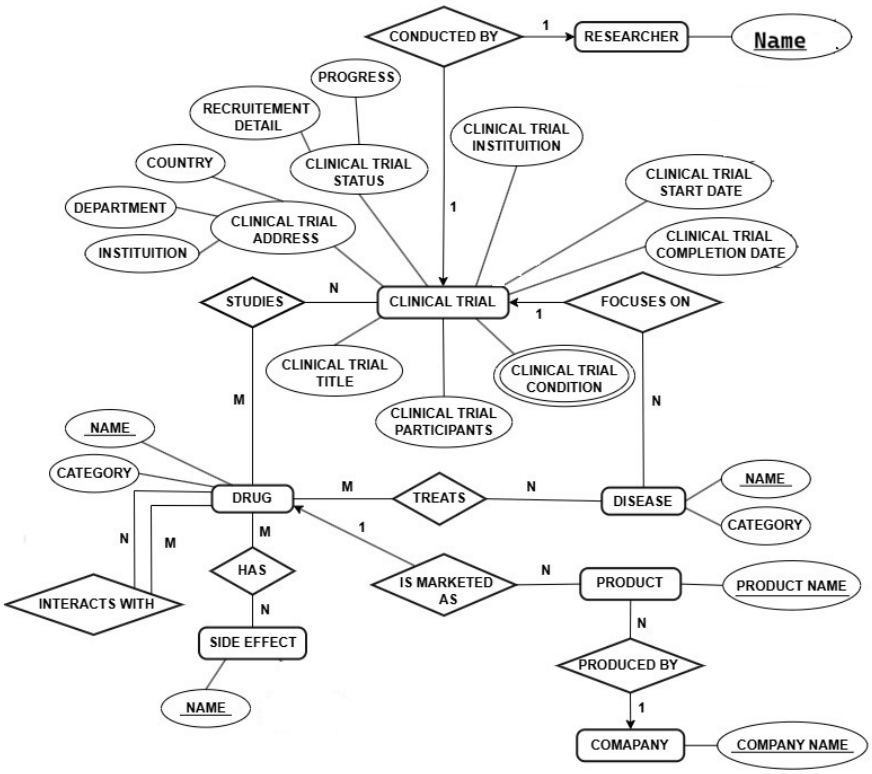
4.Drugs can interact with other Drugs (self-relationship)

5.Drugs and Products(1:N)

6.Products and company(N:1)

7.Clinical trials and Researcher(1:1)

2. Provide a list of tables with attributes that you would use to store your data.


CONDUCTED BY
1
RESEARCHER
Name
PROGRESS
RECRUITEMENT DETAIL
CLINICAL TRIAL INSTITUITION
COUNTRY
CLINICAL TRIAL STATUS
CLINICAL TRIAL START DATE
DEPARTMENT
CLINICAL TRIAL ADDRESS
1
CLINICAL TRIAL COMPLETION DATE
INSTITUITION
STUDIES
N
CLINICAL TRIAL
1
FOCUSES ON
CLINICAL TRIAL TITLE
CLINICAL TRIAL CONDITION
M
N
CLINICAL TRIAL PARTICIPANTS
NAME
CATEGORY
DRUG
M
TREATS
N
DISEASE
NAME
CATEGORY
N
M
M
1
HAS
IS MARKETED AS
N
PRODUCT
PRODUCT NAME
INTERACTS WITH
N
N
SIDE EFFECT
PRODUCED BY
NAME
1
COMAPANY
COMPANY NAME

3. In case you already have a good command of SQL, add the SQL commands that create your database.

Answer:

DATABASE

```sql
DROP DATABASE IF EXISTS PHARMA_DB;
CREATE DATABASE IF NOT EXISTS PHARMA_DB;
USE PHARMA_DB;

-- ========================
-- MAIN ENTITY TABLES
-- ========================

CREATE TABLE IF NOT EXISTS DRUG (
    DRUG_NAME        VARCHAR(255) PRIMARY KEY,
    DRUG_CATEGORY    VARCHAR(255)
);

CREATE TABLE IF NOT EXISTS DISEASE (
    DISEASE_NAME        VARCHAR(255) PRIMARY KEY,
    DISEASE_CATEGORY    VARCHAR(255)
);

CREATE TABLE IF NOT EXISTS SIDE_EFFECT (
    SIDE_EFFECT_NAME    VARCHAR(255) PRIMARY KEY
);

CREATE TABLE IF NOT EXISTS RESEARCHER (
    MAIN_RESEARCHER     VARCHAR(255) PRIMARY KEY
);

CREATE TABLE IF NOT EXISTS COMPANY (
    COMPANY_NAME        VARCHAR(255) PRIMARY KEY
);

CREATE TABLE IF NOT EXISTS PRODUCT (
    PRODUCT_NAME        VARCHAR(255) PRIMARY KEY
);

CREATE TABLE IF NOT EXISTS CLINICAL_TRIAL (
    CLINICAL_TRIAL_TITLE              VARCHAR(500) PRIMARY KEY,
    CLINICAL_TRIAL_STATUS             VARCHAR(100),
    CLINICAL_TRIAL_COUNTRY            VARCHAR(255),
    CLINICAL_TRIAL_DEPARTMENT         VARCHAR(255),
    CLINICAL_TRIAL_INSTITUTION        VARCHAR(255),
    CLINICAL_TRIAL_START_DATE         DATE,
    CLINICAL_TRIAL_COMPLETION_DATE    DATE,
    CLINICAL_TRIAL_CONDITION          VARCHAR(255),
    CLINICAL_TRIAL_PARTICIPANTS       INT
);
```

```sql
-- ========================
-- RELATIONSHIP TABLES
-- ========================
CREATE TABLE IF NOT EXISTS STUDIES (
    CLINICAL_TRIAL_TITLE   VARCHAR(500),
    DRUG_NAME              VARCHAR(255),
    PRIMARY KEY (CLINICAL_TRIAL_TITLE, DRUG_NAME),
    CONSTRAINT FK_STUDIES_TRIAL FOREIGN KEY (CLINICAL_TRIAL_TITLE)
        REFERENCES CLINICAL_TRIAL(CLINICAL_TRIAL_TITLE)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_STUDIES_DRUG FOREIGN KEY (DRUG_NAME)
        REFERENCES DRUG(DRUG_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
-- TREATS : DRUG (M) <-> (N) DISEASE
CREATE TABLE IF NOT EXISTS TREATS (
    DRUG_NAME        VARCHAR(255),
    DISEASE_NAME     VARCHAR(255),
    PRIMARY KEY (DRUG_NAME, DISEASE_NAME),
    CONSTRAINT FK_TREATS_DRUG FOREIGN KEY (DRUG_NAME)
        REFERENCES DRUG(DRUG_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_TREATS_DISEASE FOREIGN KEY (DISEASE_NAME)
        REFERENCES DISEASE(DISEASE_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
-- HAS : DRUG (M) <-> (N) SIDE_EFFECT
CREATE TABLE IF NOT EXISTS HAS_SIDE_EFFECT (
    DRUG_NAME          VARCHAR(255),
    SIDE_EFFECT_NAME   VARCHAR(255),
    PRIMARY KEY (DRUG_NAME, SIDE_EFFECT_NAME),
    CONSTRAINT FK_HAS_DRUG FOREIGN KEY (DRUG_NAME)
        REFERENCES DRUG(DRUG_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_HAS_SIDE_EFFECT FOREIGN KEY (SIDE_EFFECT_NAME)
        REFERENCES SIDE_EFFECT(SIDE_EFFECT_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
-- INTERACTS_WITH : recursive M:N on DRUG
CREATE TABLE IF NOT EXISTS INTERACTS_WITH (
    DRUG_NAME                 VARCHAR(255),
    INTERACTS_WITH_DRUG_NAME  VARCHAR(255),
    PRIMARY KEY (DRUG_NAME, INTERACTS_WITH_DRUG_NAME),
    CONSTRAINT FK_INTERACTS_DRUG1 FOREIGN KEY (DRUG_NAME)
```

```sql
        REFERENCES DRUG(DRUG_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_INTERACTS_DRUG2 FOREIGN KEY (INTERACTS_WITH_DRUG_NAME)
        REFERENCES DRUG(DRUG_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
-- CONDUCTED_BY : CLINICAL_TRIAL (1) <-> (1) RESEARCHER
CREATE TABLE IF NOT EXISTS CONDUCTED_BY (
    CLINICAL_TRIAL_TITLE   VARCHAR(500) PRIMARY KEY,
    MAIN_RESEARCHER        VARCHAR(255) UNIQUE,
    CONSTRAINT FK_CB_TRIAL FOREIGN KEY (CLINICAL_TRIAL_TITLE)
        REFERENCES CLINICAL_TRIAL(CLINICAL_TRIAL_TITLE)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_CB_RESEARCHER FOREIGN KEY (MAIN_RESEARCHER)
        REFERENCES RESEARCHER(MAIN_RESEARCHER)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
-- FOCUSES_ON : CLINICAL_TRIAL (1) -> (N) DISEASE
-- One clinical trial focuses on one disease entry here
-- A disease can appear in many trials
CREATE TABLE IF NOT EXISTS FOCUSES_ON (
    CLINICAL_TRIAL_TITLE   VARCHAR(500) PRIMARY KEY,
    DISEASE_NAME           VARCHAR(255),
    CONSTRAINT FK_FO_TRIAL FOREIGN KEY (CLINICAL_TRIAL_TITLE)
        REFERENCES CLINICAL_TRIAL(CLINICAL_TRIAL_TITLE)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_FO_DISEASE FOREIGN KEY (DISEASE_NAME)
        REFERENCES DISEASE(DISEASE_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
-- IS_MARKETED_AS : DRUG (1) -> (N) PRODUCT
-- One drug can have many products
-- Each product belongs to one drug
CREATE TABLE IF NOT EXISTS IS_MARKETED_AS (
    PRODUCT_NAME      VARCHAR(255) PRIMARY KEY,
    DRUG_NAME         VARCHAR(255),
    CONSTRAINT FK_IMA_PRODUCT FOREIGN KEY (PRODUCT_NAME)
        REFERENCES PRODUCT(PRODUCT_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_IMA_DRUG FOREIGN KEY (DRUG_NAME)
        REFERENCES DRUG(DRUG_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

```sql
-- PRODUCED_BY : PRODUCT (N) -> (1) COMPANY
CREATE TABLE IF NOT EXISTS PRODUCED_BY (
    PRODUCT_NAME      VARCHAR(255) PRIMARY KEY,
    COMPANY_NAME      VARCHAR(255),
    CONSTRAINT FK_PB_PRODUCT FOREIGN KEY (PRODUCT_NAME)
        REFERENCES PRODUCT(PRODUCT_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_PB_COMPANY FOREIGN KEY (COMPANY_NAME)
        REFERENCES COMPANY(COMPANY_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Python SCRIPT

```
import pandas as pd
import os
import mysql.connector
from mysql.connector import Error
from collections import defaultdict

# ========== CONFIGURATION ==========
# EXCEL_FILE = "sample.xlsx"
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
EXCEL_FILE = os.path.join(BASE_DIR, "sample.xlsx")
DB_CONFIG = {
    "host": "localhost",
    "user": "sheikh",           # change as needed
    "password": "miniso123",          # set your MySQL root password
    "database": "PHARMA_DB_TASK1"
}

# ========== HELPER FUNCTIONS ==========
def clean_str(val):
    """Convert to string, strip, return None if empty or NaN."""
    if pd.isna(val):
        return None
    s = str(val).strip()
    return s if s else None

def get_side_effects(row):
    """Extract non-empty side effects from columns B-F (indices 1-5)."""
    effects = []
    for i in range(1, 6):  # columns B to F
        eff = clean_str(row.iloc[i])
        if eff:
            effects.append(eff)
    return effects

def get_interacts(row):
    """Extract non-empty interacting drugs from columns G-I (indices 6-8)."""
    interacts = []
    for i in range(6, 9):
        drug = clean_str(row.iloc[i])
        if drug:
            interacts.append(drug)
    return interacts

# ========== MAIN IMPORT ==========
def main():
    # 1. Read Excel
    df = pd.read_excel(EXCEL_FILE, sheet_name="Sheet1", header=0)
    # Rename columns for easier access (optional)
    # Expected columns order:
```

```
# 0:A drug_name, 1:B side_effect1, 2:C side_effect2, 3:D side_effect3, 4:E
    side_effect4, 5:F side_effect5,
# 6:G interacts1, 7:H interacts2, 8:I interacts3, 9:J disease_name, 10:K
    disease_category,
# 11:L drug_category, 12:M product_name, 13:N company_name, 14:O
    clinical_trial_title,
# 15:P start_date, 16:Q completion_date, 17:R participants, 18:S status,
# 19:T condition1, 20:U condition2, 21:V address/country, 22:W institution,
# 23:X department, 24:Y main_researcher, 25:Z condition3
# We'll use iloc positions.

# Data containers
drugs = {}                  # drug_name -> set of categories
diseases = {}               # disease_name -> disease_category
side_effects = set()
researchers = set()
companies = set()
products = {}               # product_name -> (drug_name, company_name)
clinical_trials = {}        # trial_title -> dict of attributes
interacts_set = set()       # (drug_name, interacts_with)
treats_set = set()          # (drug_name, disease_name)
has_side_effect_set = set()  # (drug_name, side_effect)
studies_set = set()         # (trial_title, drug_name)
conducted_by_dict = {}      # trial_title -> researcher
focuses_on_dict = {}        # trial_title -> disease_name (first condition)
is_marketed_as_dict = {}    # product_name -> drug_name
produced_by_dict = {}       # product_name -> company_name

# 2. Iterate rows
for idx, row in df.iterrows():
    drug_name = clean_str(row.iloc[0])
    if not drug_name:
        # Skip rows without a drug name (they are incomplete/duplicate
            entries)
        continue

    # --- Drug & Category ---
    drug_cat = clean_str(row.iloc[11])
    if drug_cat:
        if drug_name not in drugs:
            drugs[drug_name] = set()
        drugs[drug_name].add(drug_cat)

    # --- Disease & Category ---
    disease_name = clean_str(row.iloc[9])
    disease_cat = clean_str(row.iloc[10])
    if disease_name:
        diseases[disease_name] = disease_cat
        # Treats relationship
        treats_set.add((drug_name, disease_name))
```

```
        # --- Side Effects ---
        for effect in get_side_effects(row):
            side_effects.add(effect)
            has_side_effect_set.add((drug_name, effect))

        # --- Interacts With ---
        for interact_drug in get_interacts(row):
            # Ensure the interacting drug exists in drugs set (will be added
                later)
            # We will add it to drugs with unknown category later.
            if interact_drug not in drugs:
                drugs[interact_drug] = set()  # placeholder, category unknown
            interacts_set.add((drug_name, interact_drug))

        # --- Product & Company ---
        product_name = clean_str(row.iloc[12])
        company_name = clean_str(row.iloc[13])
        if product_name and company_name:
            products[product_name] = (drug_name, company_name)
            is_marketed_as_dict[product_name] = drug_name
            produced_by_dict[product_name] = company_name
            companies.add(company_name)

        # --- Clinical Trial ---
        trial_title = clean_str(row.iloc[14])
        if trial_title:
            # Check if we already processed this trial
            if trial_title not in clinical_trials:
                # Extract trial attributes
                start = row.iloc[15]
                completion = row.iloc[16]
                participants = row.iloc[17]
                status = clean_str(row.iloc[18])
                country = clean_str(row.iloc[21])   # column V
                institution = clean_str(row.iloc[22]) # column W
                department = clean_str(row.iloc[23])  # column X
                # Convert dates
                start_date = pd.to_datetime(start).date() if pd.notna(start)
                    else None
                end_date = pd.to_datetime(completion).date() if pd.notna(
                    completion) else None
                participants_int = int(participants) if pd.notna(participants)
                    else None
                # First condition (column T) for FOCUSES_ON
                condition1 = clean_str(row.iloc[19])
                clinical_trials[trial_title] = {
                    "status": status,
                    "country": country,
                    "department": department,
                    "institution": institution,
                    "start_date": start_date,
```

```
                "completion_date": end_date,
                "condition": condition1,   # for focus
                "participants": participants_int
            }
        else:
            # If trial already exists, maybe update condition? Use first
                encountered.
            pass

        # Studies relationship
        studies_set.add((trial_title, drug_name))

        # Conducted By (researcher)
        researcher = clean_str(row.iloc[24])
        if researcher:
            researchers.add(researcher)
            conducted_by_dict[trial_title] = researcher

        # Focuses On (use condition from column T, only if not already set)
        # Use the first condition we see; if already set, keep original.
        if trial_title not in focuses_on_dict:
            cond = clean_str(row.iloc[19])
            if cond:
                focuses_on_dict[trial_title] = cond

# 3. Insert into MySQL
try:
    conn = mysql.connector.connect(**DB_CONFIG)
    cursor = conn.cursor()
    # Disable foreign key checks for bulk insert order
    cursor.execute("SET FOREIGN_KEY_CHECKS = 0")

    # ---- Insert DRUGS (with concatenated categories) ----
    for drug_name, cat_set in drugs.items():
        cat_str = ", ".join(sorted(cat_set)) if cat_set else None
        cursor.execute(
            "INSERT IGNORE INTO DRUG (DRUG_NAME, DRUG_CATEGORY) VALUES (%s,
                %s)",
            (drug_name, cat_str)
        )

    # ---- Insert DISEASE ----
    for disease_name, disease_cat in diseases.items():
        cursor.execute(
            "INSERT IGNORE INTO DISEASE (DISEASE_NAME, DISEASE_CATEGORY)
                VALUES (%s, %s)",
            (disease_name, disease_cat)
        )

    # ---- Insert SIDE_EFFECT ----
    for effect in side_effects:
```

```
            cursor.execute(
                "INSERT IGNORE INTO SIDE_EFFECT (SIDE_EFFECT_NAME) VALUES (%s)"
                    ,
                (effect,)
            )

        # ---- Insert RESEARCHER ----
        for researcher in researchers:
            cursor.execute(
                "INSERT IGNORE INTO RESEARCHER (MAIN_RESEARCHER) VALUES (%s)",
                (researcher,)
            )

        # ---- Insert COMPANY ----
        for company in companies:
            cursor.execute(
                "INSERT IGNORE INTO COMPANY (COMPANY_NAME) VALUES (%s)",
                (company,)
            )

        # ---- Insert PRODUCT ----
        for product_name in products.keys():
            cursor.execute(
                "INSERT IGNORE INTO PRODUCT (PRODUCT_NAME) VALUES (%s)",
                (product_name,)
            )

        # ---- Insert CLINICAL_TRIAL ----
        for title, attrs in clinical_trials.items():
            cursor.execute("""
                INSERT IGNORE INTO CLINICAL_TRIAL (
                    CLINICAL_TRIAL_TITLE, CLINICAL_TRIAL_STATUS,
                        CLINICAL_TRIAL_COUNTRY,
                    CLINICAL_TRIAL_DEPARTMENT, CLINICAL_TRIAL_INSTITUTION,
                    CLINICAL_TRIAL_START_DATE, CLINICAL_TRIAL_COMPLETION_DATE,
                    CLINICAL_TRIAL_CONDITION, CLINICAL_TRIAL_PARTICIPANTS
                ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
            """, (
                title, attrs["status"], attrs["country"],
                attrs["department"], attrs["institution"],
                attrs["start_date"], attrs["completion_date"],
                attrs["condition"], attrs["participants"]
            ))

        # ---- Relationship tables ----
        # TREATS
        for drug_name, disease_name in treats_set:
            cursor.execute(
                "INSERT IGNORE INTO TREATS (DRUG_NAME, DISEASE_NAME) VALUES (%s
                    , %s)",
                (drug_name, disease_name)
```

```
        )

    # HAS_SIDE_EFFECT
    for drug_name, effect in has_side_effect_set:
        cursor.execute(
            "INSERT IGNORE INTO HAS_SIDE_EFFECT (DRUG_NAME,
                SIDE_EFFECT_NAME) VALUES (%s, %s)",
            (drug_name, effect)
        )

    # INTERACTS_WITH
    for drug_name, interact_drug in interacts_set:
        cursor.execute(
            "INSERT IGNORE INTO INTERACTS_WITH (DRUG_NAME,
                INTERACTS_WITH_DRUG_NAME) VALUES (%s, %s)",
            (drug_name, interact_drug)
        )

    # STUDIES
    for trial_title, drug_name in studies_set:
        cursor.execute(
            "INSERT IGNORE INTO STUDIES (CLINICAL_TRIAL_TITLE, DRUG_NAME)
                VALUES (%s, %s)",
            (trial_title, drug_name)
        )

    # CONDUCTED_BY
    for trial_title, researcher in conducted_by_dict.items():
        cursor.execute(
            "INSERT IGNORE INTO CONDUCTED_BY (CLINICAL_TRIAL_TITLE,
                MAIN_RESEARCHER) VALUES (%s, %s)",
            (trial_title, researcher)
        )

    # FOCUSES_ON
    for trial_title, disease_name in focuses_on_dict.items():
        cursor.execute(
            "INSERT IGNORE INTO FOCUSES_ON (CLINICAL_TRIAL_TITLE,
                DISEASE_NAME) VALUES (%s, %s)",
            (trial_title, disease_name)
        )

    # IS_MARKETED_AS
    for product_name, drug_name in is_marketed_as_dict.items():
        cursor.execute(
            "INSERT IGNORE INTO IS_MARKETED_AS (PRODUCT_NAME, DRUG_NAME)
                VALUES (%s, %s)",
            (product_name, drug_name)
        )

    # PRODUCED_BY
```

```
        for product_name, company_name in produced_by_dict.items():
            cursor.execute(
                "INSERT IGNORE INTO PRODUCED_BY (PRODUCT_NAME, COMPANY_NAME)
                    VALUES (%s, %s)",
                (product_name, company_name)
            )

        # Re-enable foreign key checks
        cursor.execute("SET FOREIGN_KEY_CHECKS = 1")
        conn.commit()
        print("Data imported successfully!")

    except Error as e:
        print(f"Error: {e}")
        conn.rollback()
    finally:
        if conn.is_connected():
            cursor.close()
            conn.close()

if __name__ == "__main__":
    main()
```

1 Task 2.1 — Relevant Data Identification

1.1 Entities and Key Attributes

Entity	Key Attributes
DRUG	drug_name (PK), drug_category
DISEASE	disease_name (PK), disease_category
SIDE_EFFECT	side_effect_name (PK)
RESEARCHER	researcher_name (PK)
PRODUCT	product_name (PK), company_name, drug_name (FK)
CLINICAL TRIAL	clinical_trial_title (PK), status, start_date, completion_date, participants, country, institution, department, researcher_name (FK)

1.2 Multivalued Attributes

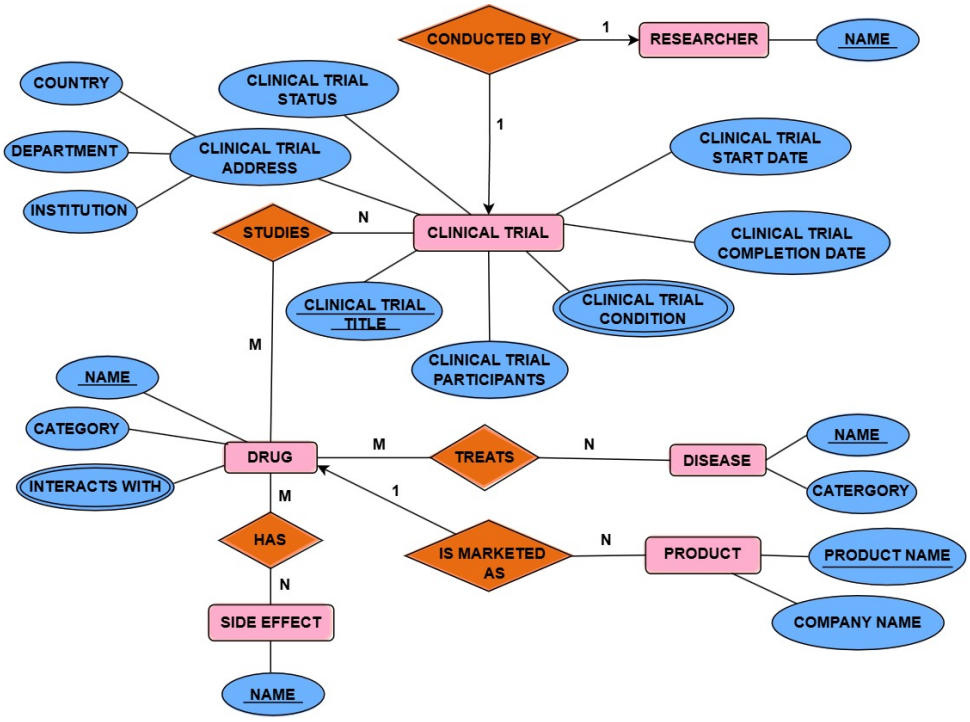
- DRUG — INTERACTS WITH
- CLINICAL TRIAL — CONDITION

1.3 Relationships and Cardinalities

Relationship	Entities	Cardinality	Min-Max

STUDIES	DRUG — CLINICAL TRIAL	M:N	[0,N] — [0,N]
TREATS	DRUG — DISEASE	M:N	[0,N] — [0,N]
HAS (side effect)	DRUG — SIDE EFFECT	M:N	[0,N] — [1,N]
IS MARKETED AS	DRUG — PRODUCT	1:N	[0,N] — [1,1]
CONDUCTED BY	CLINICAL TRIAL — RESEARCHER	1:1	[1,1] — [0,1]

2 Task 2.1 — ER Diagram



2.1 Design Choices

- DRUG INTERACTION is a recursive M:N relationship
- PRODUCT includes company name
- Multivalued attributes handled separately
- No synthetic ID attributes
- CLINICAL TRIAL CONDITION is multivalued

3 Task 2.3 — Relational Schema

3.1 Entity Relations

```
DRUG(drug_name PK, drug_category)
```

RESEARCHER(researcher_name PK)
DISEASE disease_name (PK), disease_category
PRODUCT product_name (PK), company_name, drug_name (FK -> DRUG)
CLINICAL_TRIAL
clinical_trial_title (PK), clinical_trial_status,
clinical_trial_start_date, clinical_trial_completion_date,
clinical_trial_participants, trial_country, trial_institution,
trial_department, researcher_name (FK -> RESEARCHER, UNIQUE)
CLINICAL_TRIAL_CONDITIONclinical_trial_title (PK,
FK->CLINICAL_TRIAL),
condition_name (PK) (Multivalued attribute of CLINICAL TRIAL)

3.2 SQL DDL

```
DROP DATABASE IF EXISTS PHARMA_DB_TASK1;
CREATE DATABASE IF NOT EXISTS PHARMA_DB_TASK1;
USE PHARMA_DB_TASK1;

CREATE TABLE IF NOT EXISTS DRUG (
    DRUG_NAME       VARCHAR(255) PRIMARY KEY,
    DRUG_CATEGORY   VARCHAR(255)
);

CREATE TABLE IF NOT EXISTS DISEASE (
    DISEASE_NAME       VARCHAR(255) PRIMARY KEY,
    DISEASE_CATEGORY   VARCHAR(255)
);

CREATE TABLE IF NOT EXISTS SIDE_EFFECT (
    SIDE_EFFECT_NAME   VARCHAR(255) PRIMARY KEY
);

CREATE TABLE IF NOT EXISTS RESEARCHER (
    MAIN_RESEARCHER    VARCHAR(255) PRIMARY KEY
);

CREATE TABLE IF NOT EXISTS COMPANY (
    COMPANY_NAME       VARCHAR(255) PRIMARY KEY
);

CREATE TABLE IF NOT EXISTS PRODUCT (
    PRODUCT_NAME       VARCHAR(255) PRIMARY KEY
);

CREATE TABLE IF NOT EXISTS CLINICAL_TRIAL (
    CLINICAL_TRIAL_TITLE             VARCHAR(500) PRIMARY KEY,
    CLINICAL_TRIAL_STATUS            VARCHAR(100),
    CLINICAL_TRIAL_COUNTRY           VARCHAR(255),
    CLINICAL_TRIAL_DEPARTMENT        VARCHAR(255),
    CLINICAL_TRIAL_INSTITUTION       VARCHAR(255),
```

```sql
    CLINICAL_TRIAL_START_DATE        DATE,
    CLINICAL_TRIAL_COMPLETION_DATE   DATE,
    CLINICAL_TRIAL_CONDITION         VARCHAR(255),
    CLINICAL_TRIAL_PARTICIPANTS      INT
);
CREATE TABLE IF NOT EXISTS STUDIES (
    CLINICAL_TRIAL_TITLE   VARCHAR(500),
    DRUG_NAME              VARCHAR(255),
    PRIMARY KEY (CLINICAL_TRIAL_TITLE, DRUG_NAME),
    CONSTRAINT FK_STUDIES_TRIAL FOREIGN KEY (CLINICAL_TRIAL_TITLE)
        REFERENCES CLINICAL_TRIAL(CLINICAL_TRIAL_TITLE)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_STUDIES_DRUG FOREIGN KEY (DRUG_NAME)
        REFERENCES DRUG(DRUG_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS TREATS (
    DRUG_NAME        VARCHAR(255),
    DISEASE_NAME     VARCHAR(255),
    PRIMARY KEY (DRUG_NAME, DISEASE_NAME),
    CONSTRAINT FK_TREATS_DRUG FOREIGN KEY (DRUG_NAME)
        REFERENCES DRUG(DRUG_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_TREATS_DISEASE FOREIGN KEY (DISEASE_NAME)
        REFERENCES DISEASE(DISEASE_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS HAS_SIDE_EFFECT (
    DRUG_NAME          VARCHAR(255),
    SIDE_EFFECT_NAME   VARCHAR(255),
    PRIMARY KEY (DRUG_NAME, SIDE_EFFECT_NAME),
    CONSTRAINT FK_HAS_DRUG FOREIGN KEY (DRUG_NAME)
        REFERENCES DRUG(DRUG_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_HAS_SIDE_EFFECT FOREIGN KEY (SIDE_EFFECT_NAME)
        REFERENCES SIDE_EFFECT(SIDE_EFFECT_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS INTERACTS_WITH (
    DRUG_NAME                 VARCHAR(255),
    INTERACTS_WITH_DRUG_NAME  VARCHAR(255),
    PRIMARY KEY (DRUG_NAME, INTERACTS_WITH_DRUG_NAME),
```

```sql
    CONSTRAINT FK_INTERACTS_DRUG1 FOREIGN KEY (DRUG_NAME)
        REFERENCES DRUG(DRUG_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_INTERACTS_DRUG2 FOREIGN KEY (INTERACTS_WITH_DRUG_NAME)
        REFERENCES DRUG(DRUG_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS CONDUCTED_BY (
    CLINICAL_TRIAL_TITLE   VARCHAR(500) PRIMARY KEY,
    MAIN_RESEARCHER        VARCHAR(255) UNIQUE,
    CONSTRAINT FK_CB_TRIAL FOREIGN KEY (CLINICAL_TRIAL_TITLE)
        REFERENCES CLINICAL_TRIAL(CLINICAL_TRIAL_TITLE)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_CB_RESEARCHER FOREIGN KEY (MAIN_RESEARCHER)
        REFERENCES RESEARCHER(MAIN_RESEARCHER)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS FOCUSES_ON (
    CLINICAL_TRIAL_TITLE   VARCHAR(500) PRIMARY KEY,
    DISEASE_NAME           VARCHAR(255),
    CONSTRAINT FK_FO_TRIAL FOREIGN KEY (CLINICAL_TRIAL_TITLE)
        REFERENCES CLINICAL_TRIAL(CLINICAL_TRIAL_TITLE)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_FO_DISEASE FOREIGN KEY (DISEASE_NAME)
        REFERENCES DISEASE(DISEASE_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS IS_MARKETED_AS (
    PRODUCT_NAME      VARCHAR(255) PRIMARY KEY,
    DRUG_NAME         VARCHAR(255),
    CONSTRAINT FK_IMA_PRODUCT FOREIGN KEY (PRODUCT_NAME)
        REFERENCES PRODUCT(PRODUCT_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_IMA_DRUG FOREIGN KEY (DRUG_NAME)
        REFERENCES DRUG(DRUG_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS PRODUCED_BY (
    PRODUCT_NAME      VARCHAR(255) PRIMARY KEY,
```

```
    COMPANY_NAME      VARCHAR(255),
    CONSTRAINT FK_PB_PRODUCT FOREIGN KEY (PRODUCT_NAME)
        REFERENCES PRODUCT(PRODUCT_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT FK_PB_COMPANY FOREIGN KEY (COMPANY_NAME)
        REFERENCES COMPANY(COMPANY_NAME)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

4 Task 4 — Comparison with Task 1

Aspect	Task 1	Task 2
COMPANY	Separate table	Merged into PRODUCT
Interects with	Recursive relation	Merged into drug as composite attribute
Clinical Trials and Disease	had focus on relation	got rid of the relation

5 SQL Queries

5.1 Query (a)

```
USE PHARMA_DB_TASK2;

SELECT COUNT(DISTINCT drug_name) AS number_of_drugs
FROM HAS_SIDE_EFFECT
WHERE side_effect_name = 'nausea';
```

5.2 Query (b)

```
USE PHARMA_DB_TASK2;

SELECT DISTINCT drug_name
FROM DRUG_INTERACTION
WHERE LOWER(interacts_with_drug_name) = 'butabarbital'
ORDER BY drug_name;
```

5.3 Query (c)

```
USE PHARMA_DB_TASK2;

SELECT drug_name
FROM HAS_SIDE_EFFECT
WHERE LOWER(side_effect_name) IN ('cough', 'headache')
```

```
GROUP BY drug_name
HAVING COUNT(DISTINCT LOWER(side_effect_name)) = 2
ORDER BY drug_name;
```

5.4 Query (d)

```
USE PHARMA_DB_TASK2;

SELECT DISTINCT t.drug_name
FROM TREATS t
JOIN DISEASE d
  ON t.disease_name = d.disease_name
WHERE LOWER(d.disease_category) = 'endocrine'
ORDER BY t.drug_name;
```

5.5 Query (e)

```
USE PHARMA_DB_TASK2;

SELECT t.drug_name, COUNT(*) AS usage_count
FROM TREATS t
JOIN DISEASE d
  ON t.disease_name = d.disease_name
WHERE LOWER(d.disease_category) = 'immunological'
  AND t.drug_name NOT IN (
      SELECT t2.drug_name
      FROM TREATS t2
      JOIN DISEASE d2
        ON t2.disease_name = d2.disease_name
      WHERE LOWER(d2.disease_category) = 'hematological'
  )
GROUP BY t.drug_name
ORDER BY usage_count DESC, t.drug_name
LIMIT 1;
```

5.6 Query (f)

```
USE PHARMA_DB_TASK2;

SELECT disease_name
FROM TREATS
WHERE drug_name = 'hydrocortisone'
  AND disease_name NOT IN (
      SELECT disease_name
      FROM TREATS
      WHERE drug_name = 'etanercept'
  )
ORDER BY disease_name;
```

5.7 Query (g)

```sql
USE PHARMA_DB_TASK2;

SELECT h.side_effect_name, COUNT(*) AS frequency
FROM HAS_SIDE_EFFECT h
JOIN TREATS t
  ON h.drug_name = t.drug_name
JOIN DISEASE d
  ON t.disease_name = d.disease_name
WHERE LOWER(d.disease_name) LIKE '%asthma%'
GROUP BY h.side_effect_name
ORDER BY frequency DESC, h.side_effect_name
LIMIT 10;
```

5.8 Query (h)

```sql
USE PHARMA_DB_TASK2;

SELECT s.drug_name
FROM STUDIES s
JOIN CLINICAL_TRIAL c
  ON s.clinical_trial_title = c.clinical_trial_title
WHERE c.clinical_trial_participants > 30
GROUP BY s.drug_name
HAVING COUNT(DISTINCT s.clinical_trial_title) > 3
ORDER BY s.drug_name;
```

5.9 Query (i)

```sql
USE PHARMA_DB_TASK2;

WITH boundaries AS (
    SELECT clinical_trial_start_date AS dt FROM CLINICAL_TRIAL
    UNION
    SELECT clinical_trial_completion_date AS dt FROM CLINICAL_TRIAL
),
ordered_boundaries AS (
    SELECT
        dt AS period_start,
        LEAD(dt) OVER (ORDER BY dt) AS period_end
    FROM boundaries
),
active_trials AS (
    SELECT
        ob.period_start,
        ob.period_end,
        c.clinical_trial_title
    FROM ordered_boundaries ob
```

```
    JOIN CLINICAL_TRIAL c
      ON ob.period_end IS NOT NULL
     AND c.clinical_trial_start_date <= ob.period_start
     AND c.clinical_trial_completion_date >= ob.period_end
),
period_counts AS (
    SELECT
        period_start,
        period_end,
        COUNT(DISTINCT clinical_trial_title) AS active_trial_count
    FROM active_trials
    GROUP BY period_start, period_end
),
max_count AS (
    SELECT MAX(active_trial_count) AS mx
    FROM period_counts
)
SELECT
    pc.period_start,
    pc.period_end,
    pc.active_trial_count,
    GROUP_CONCAT(DISTINCT s.drug_name ORDER BY s.drug_name SEPARATOR ', ') AS
        studied_drugs
FROM period_counts pc
JOIN max_count mc
  ON pc.active_trial_count = mc.mx
JOIN active_trials a
  ON a.period_start = pc.period_start
 AND a.period_end = pc.period_end
JOIN STUDIES s
  ON s.clinical_trial_title = a.clinical_trial_title
GROUP BY pc.period_start, pc.period_end, pc.active_trial_count
ORDER BY pc.period_start;
```

5.10 Query (j)

```
USE PHARMA_DB_TASK2;

SELECT DISTINCT c.researcher_name
FROM CLINICAL_TRIAL c
JOIN STUDIES s
  ON c.clinical_trial_title = s.clinical_trial_title
WHERE s.drug_name IN (
    SELECT t.drug_name
    FROM TREATS t
    JOIN DISEASE d
      ON t.disease_name = d.disease_name
    WHERE LOWER(d.disease_category) IN ('respiratory', 'cardiovascular')
    GROUP BY t.drug_name
    HAVING COUNT(DISTINCT LOWER(d.disease_category)) = 2
```

```
)
ORDER BY c.researcher_name;
```

5.11 Query (k)

```
USE PHARMA_DB_TASK2;

SELECT c.researcher_name, COUNT(DISTINCT c.clinical_trial_title) AS
    total_trials
FROM CLINICAL_TRIAL c
JOIN STUDIES s
  ON c.clinical_trial_title = s.clinical_trial_title
WHERE s.drug_name IN (
    SELECT t.drug_name
    FROM TREATS t
    JOIN DISEASE d
      ON t.disease_name = d.disease_name
    WHERE LOWER(d.disease_category) IN ('respiratory', 'cardiovascular')
    GROUP BY t.drug_name
    HAVING COUNT(DISTINCT LOWER(d.disease_category)) = 2
)
GROUP BY c.researcher_name
ORDER BY total_trials DESC, c.researcher_name
LIMIT 3;
```

5.12 Query (l)

```
USE PHARMA_DB_TASK2;

SELECT DISTINCT d.drug_category
FROM DRUG d
WHERE d.drug_name IN (
    SELECT s.drug_name
    FROM STUDIES s
    JOIN CLINICAL_TRIAL c
      ON s.clinical_trial_title = c.clinical_trial_title
    GROUP BY s.drug_name
    HAVING SUM(CASE WHEN c.trial_country <> 'United States' THEN 1 ELSE 0 END)
       = 0
)
ORDER BY d.drug_category;
```

6 Data Import Script Overview

```
import pandas as pd
import os
import mysql.connector
```

```
from mysql.connector import Error
from collections import defaultdict

# ========== CONFIGURATION ==========
# EXCEL_FILE = "sample.xlsx"
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
EXCEL_FILE = os.path.join(BASE_DIR, "sample.xlsx")
DB_CONFIG = {
    "host": "localhost",
    "user": "sheikh",          # change as needed
    "password": "miniso123",          # set your MySQL root password
    "database": "PHARMA_DB_TASK1"
}

# ========== HELPER FUNCTIONS ==========
def clean_str(val):
    """Convert to string, strip, return None if empty or NaN."""
    if pd.isna(val):
        return None
    s = str(val).strip()
    return s if s else None

def get_side_effects(row):
    """Extract non-empty side effects from columns B-F (indices 1-5)."""
    effects = []
    for i in range(1, 6):  # columns B to F
        eff = clean_str(row.iloc[i])
        if eff:
            effects.append(eff)
    return effects

def get_interacts(row):
    """Extract non-empty interacting drugs from columns G-I (indices 6-8)."""
    interacts = []
    for i in range(6, 9):
        drug = clean_str(row.iloc[i])
        if drug:
            interacts.append(drug)
    return interacts

# ========== MAIN IMPORT ==========
def main():
    # 1. Read Excel
    df = pd.read_excel(EXCEL_FILE, sheet_name="Sheet1", header=0)
    # Rename columns for easier access (optional)
    # Expected columns order:
    # 0:A drug_name, 1:B side_effect1, 2:C side_effect2, 3:D side_effect3, 4:E
        side_effect4, 5:F side_effect5,
    # 6:G interacts1, 7:H interacts2, 8:I interacts3, 9:J disease_name, 10:K
        disease_category,
```

```
    # 11:L drug_category, 12:M product_name, 13:N company_name, 14:O
        clinical_trial_title,
    # 15:P start_date, 16:Q completion_date, 17:R participants, 18:S status,
    # 19:T condition1, 20:U condition2, 21:V address/country, 22:W institution,
    # 23:X department, 24:Y main_researcher, 25:Z condition3
    # We'll use iloc positions.

    # Data containers
    drugs = {}                  # drug_name -> set of categories
    diseases = {}               # disease_name -> disease_category
    side_effects = set()
    researchers = set()
    companies = set()
    products = {}               # product_name -> (drug_name, company_name)
    clinical_trials = {}        # trial_title -> dict of attributes
    interacts_set = set()       # (drug_name, interacts_with)
    treats_set = set()          # (drug_name, disease_name)
    has_side_effect_set = set()  # (drug_name, side_effect)
    studies_set = set()         # (trial_title, drug_name)
    conducted_by_dict = {}      # trial_title -> researcher
    focuses_on_dict = {}        # trial_title -> disease_name (first condition)
    is_marketed_as_dict = {}    # product_name -> drug_name
    produced_by_dict = {}       # product_name -> company_name

    # 2. Iterate rows
    for idx, row in df.iterrows():
        drug_name = clean_str(row.iloc[0])
        if not drug_name:
            # Skip rows without a drug name (they are incomplete/duplicate
                entries)
            continue

        # --- Drug & Category ---
        drug_cat = clean_str(row.iloc[11])
        if drug_cat:
            if drug_name not in drugs:
                drugs[drug_name] = set()
            drugs[drug_name].add(drug_cat)

        # --- Disease & Category ---
        disease_name = clean_str(row.iloc[9])
        disease_cat = clean_str(row.iloc[10])
        if disease_name:
            diseases[disease_name] = disease_cat
            # Treats relationship
            treats_set.add((drug_name, disease_name))

        # --- Side Effects ---
        for effect in get_side_effects(row):
            side_effects.add(effect)
            has_side_effect_set.add((drug_name, effect))
```

```
        # --- Interacts With ---
        for interact_drug in get_interacts(row):
            # Ensure the interacting drug exists in drugs set (will be added
                later)
            # We will add it to drugs with unknown category later.
            if interact_drug not in drugs:
                drugs[interact_drug] = set()  # placeholder, category unknown
            interacts_set.add((drug_name, interact_drug))

        # --- Product & Company ---
        product_name = clean_str(row.iloc[12])
        company_name = clean_str(row.iloc[13])
        if product_name and company_name:
            products[product_name] = (drug_name, company_name)
            is_marketed_as_dict[product_name] = drug_name
            produced_by_dict[product_name] = company_name
            companies.add(company_name)

        # --- Clinical Trial ---
        trial_title = clean_str(row.iloc[14])
        if trial_title:
            # Check if we already processed this trial
            if trial_title not in clinical_trials:
                # Extract trial attributes
                start = row.iloc[15]
                completion = row.iloc[16]
                participants = row.iloc[17]
                status = clean_str(row.iloc[18])
                country = clean_str(row.iloc[21])   # column V
                institution = clean_str(row.iloc[22]) # column W
                department = clean_str(row.iloc[23])  # column X
                # Convert dates
                start_date = pd.to_datetime(start).date() if pd.notna(start)
                    else None
                end_date = pd.to_datetime(completion).date() if pd.notna(
                    completion) else None
                participants_int = int(participants) if pd.notna(participants)
                    else None
                # First condition (column T) for FOCUSES_ON
                condition1 = clean_str(row.iloc[19])
                clinical_trials[trial_title] = {
                    "status": status,
                    "country": country,
                    "department": department,
                    "institution": institution,
                    "start_date": start_date,
                    "completion_date": end_date,
                    "condition": condition1,   # for focus
                    "participants": participants_int
                }
```

```
            else:
                # If trial already exists, maybe update condition? Use first
                    encountered.
                pass

            # Studies relationship
            studies_set.add((trial_title, drug_name))

            # Conducted By (researcher)
            researcher = clean_str(row.iloc[24])
            if researcher:
                researchers.add(researcher)
                conducted_by_dict[trial_title] = researcher

            # Focuses On (use condition from column T, only if not already set)
            # Use the first condition we see; if already set, keep original.
            if trial_title not in focuses_on_dict:
                cond = clean_str(row.iloc[19])
                if cond:
                    focuses_on_dict[trial_title] = cond

    # 3. Insert into MySQL
    try:
        conn = mysql.connector.connect(**DB_CONFIG)
        cursor = conn.cursor()
        # Disable foreign key checks for bulk insert order
        cursor.execute("SET FOREIGN_KEY_CHECKS = 0")

        # ---- Insert DRUGS (with concatenated categories) ----
        for drug_name, cat_set in drugs.items():
            cat_str = ", ".join(sorted(cat_set)) if cat_set else None
            cursor.execute(
                "INSERT IGNORE INTO DRUG (DRUG_NAME, DRUG_CATEGORY) VALUES (%s,
                    %s)",
                (drug_name, cat_str)
            )

        # ---- Insert DISEASE ----
        for disease_name, disease_cat in diseases.items():
            cursor.execute(
                "INSERT IGNORE INTO DISEASE (DISEASE_NAME, DISEASE_CATEGORY)
                    VALUES (%s, %s)",
                (disease_name, disease_cat)
            )

        # ---- Insert SIDE_EFFECT ----
        for effect in side_effects:
            cursor.execute(
                "INSERT IGNORE INTO SIDE_EFFECT (SIDE_EFFECT_NAME) VALUES (%s)"
                    ,
                (effect,)
```

```
        )

    # ---- Insert RESEARCHER ----
    for researcher in researchers:
        cursor.execute(
            "INSERT IGNORE INTO RESEARCHER (MAIN_RESEARCHER) VALUES (%s)",
            (researcher,)
        )

    # ---- Insert COMPANY ----
    for company in companies:
        cursor.execute(
            "INSERT IGNORE INTO COMPANY (COMPANY_NAME) VALUES (%s)",
            (company,)
        )

    # ---- Insert PRODUCT ----
    for product_name in products.keys():
        cursor.execute(
            "INSERT IGNORE INTO PRODUCT (PRODUCT_NAME) VALUES (%s)",
            (product_name,)
        )

    # ---- Insert CLINICAL_TRIAL ----
    for title, attrs in clinical_trials.items():
        cursor.execute("""
            INSERT IGNORE INTO CLINICAL_TRIAL (
                CLINICAL_TRIAL_TITLE, CLINICAL_TRIAL_STATUS,
                    CLINICAL_TRIAL_COUNTRY,
                CLINICAL_TRIAL_DEPARTMENT, CLINICAL_TRIAL_INSTITUTION,
                CLINICAL_TRIAL_START_DATE, CLINICAL_TRIAL_COMPLETION_DATE,
                CLINICAL_TRIAL_CONDITION, CLINICAL_TRIAL_PARTICIPANTS
            ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
        """, (
            title, attrs["status"], attrs["country"],
            attrs["department"], attrs["institution"],
            attrs["start_date"], attrs["completion_date"],
            attrs["condition"], attrs["participants"]
        ))

    # ---- Relationship tables ----
    # TREATS
    for drug_name, disease_name in treats_set:
        cursor.execute(
            "INSERT IGNORE INTO TREATS (DRUG_NAME, DISEASE_NAME) VALUES (%s
                , %s)",
            (drug_name, disease_name)
        )

    # HAS_SIDE_EFFECT
    for drug_name, effect in has_side_effect_set:
```

```
            cursor.execute(
                "INSERT IGNORE INTO HAS_SIDE_EFFECT (DRUG_NAME,
                    SIDE_EFFECT_NAME) VALUES (%s, %s)",
                (drug_name, effect)
            )

        # INTERACTS_WITH
        for drug_name, interact_drug in interacts_set:
            cursor.execute(
                "INSERT IGNORE INTO INTERACTS_WITH (DRUG_NAME,
                    INTERACTS_WITH_DRUG_NAME) VALUES (%s, %s)",
                (drug_name, interact_drug)
            )

        # STUDIES
        for trial_title, drug_name in studies_set:
            cursor.execute(
                "INSERT IGNORE INTO STUDIES (CLINICAL_TRIAL_TITLE, DRUG_NAME)
                    VALUES (%s, %s)",
                (trial_title, drug_name)
            )

        # CONDUCTED_BY
        for trial_title, researcher in conducted_by_dict.items():
            cursor.execute(
                "INSERT IGNORE INTO CONDUCTED_BY (CLINICAL_TRIAL_TITLE,
                    MAIN_RESEARCHER) VALUES (%s, %s)",
                (trial_title, researcher)
            )

        # FOCUSES_ON
        for trial_title, disease_name in focuses_on_dict.items():
            cursor.execute(
                "INSERT IGNORE INTO FOCUSES_ON (CLINICAL_TRIAL_TITLE,
                    DISEASE_NAME) VALUES (%s, %s)",
                (trial_title, disease_name)
            )

        # IS_MARKETED_AS
        for product_name, drug_name in is_marketed_as_dict.items():
            cursor.execute(
                "INSERT IGNORE INTO IS_MARKETED_AS (PRODUCT_NAME, DRUG_NAME)
                    VALUES (%s, %s)",
                (product_name, drug_name)
            )

        # PRODUCED_BY
        for product_name, company_name in produced_by_dict.items():
            cursor.execute(
                "INSERT IGNORE INTO PRODUCED_BY (PRODUCT_NAME, COMPANY_NAME)
                    VALUES (%s, %s)",
```

```
                (product_name, company_name)
            )

        # Re-enable foreign key checks
        cursor.execute("SET FOREIGN_KEY_CHECKS = 1")
        conn.commit()
        print("Data imported successfully!")

    except Error as e:
        print(f"Error: {e}")
        conn.rollback()
    finally:
        if conn.is_connected():
            cursor.close()
            conn.close()

if __name__ == "__main__":
    main()
```